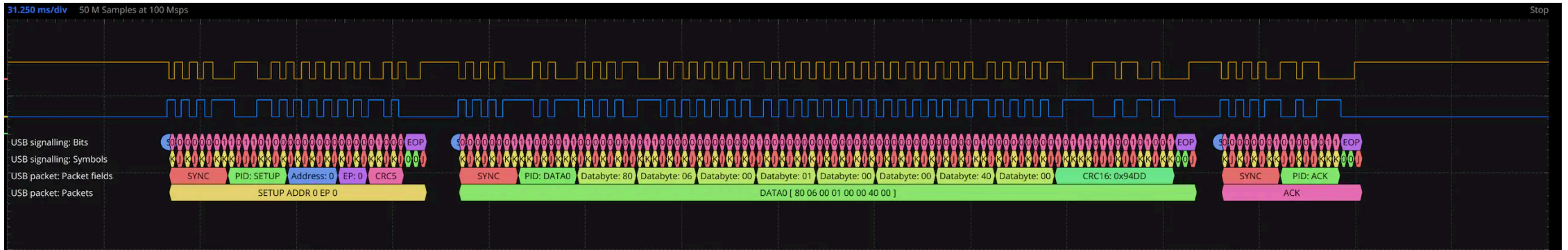


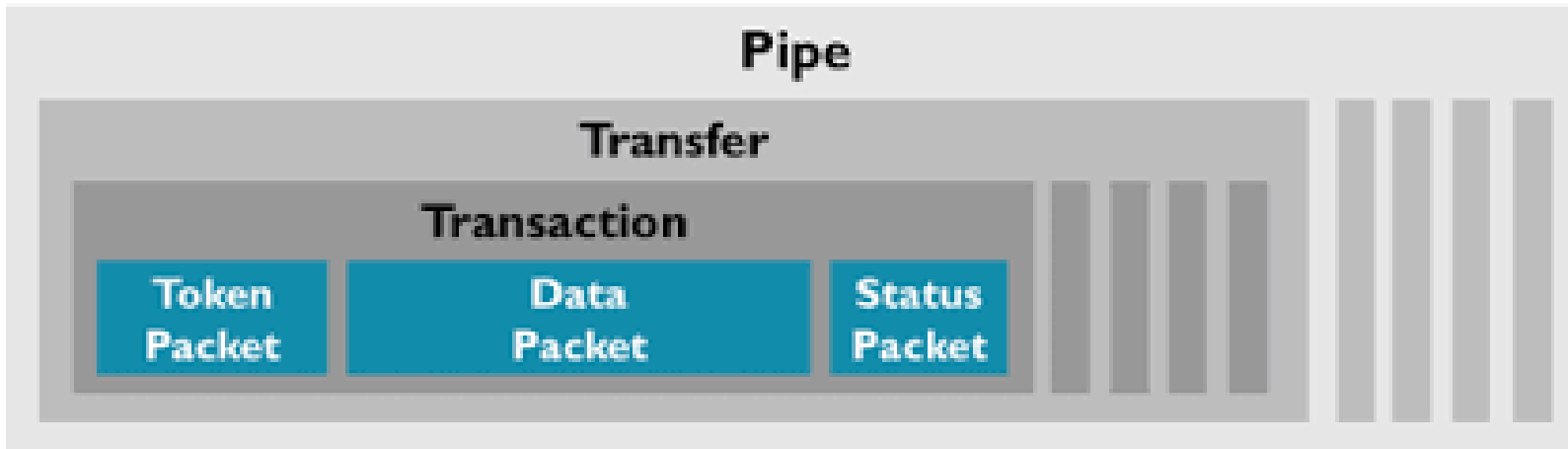
USB Part 3 (Protocols)



USB is more than just a cable — it's a complex communication protocol with multiple layers and concepts that work together to make our devices talk to each other seamlessly.

- USB Packets/Transactions/Transfers/Pipes/Frames
- USB Protocol Stack Layers

From Packets to Frames



USB communication uses packet based transactions structured into transfers, which are sent through logical pipes and scheduled in frames.

Packet ← smallest unit (SYNC + PID + data + CRC)
 ↓
Transaction ← 3 packets: Token + Data + Handshake
 ↓
Transfer ← one or more transactions (e.g., read 512 bytes)
 ↓
Pipe ← logical channel connecting host <-> endpoint
 ↓
Frame ← 1ms time window; all bus activity fits inside

Each level builds on the one below. Let's go through each.

Packets

Every USB communication is broken into packets:

Example: Reading 64 bytes from a flash drive

Packet 1 (TOKEN):

SYNC 0x80	PID IN	ADDRESS 3	ENDPOINT 2	CRC ...
--------------	-----------	--------------	---------------	------------

- SYNC: signals the start of a packet
- PID: Packet ID (what type of packet is this? TOKEN, DATA, HANDSHAKE)
- ADDRESS: which device (0–127)
- ENDPOINT: which endpoint (0–15): During enumeration host knows that device #3 has endpoint 2 for data input.
- CRC: error checking

Host sends TOKEN to address 3, endpoint 2 → "Device #3, endpoint 2: Send me data!"

Packet 2 (DATA):

SYNC 0x80	DATA0	64 bytes of file data	CRC ...
--------------	-------	--------------------------	------------

Now, device #3 responds with a DATA packet containing 64 bytes of file data from the flash drive.

"Device sends the requested data"

Packet 3 (HANDSHAKE):

SYNC 0x80	PID ACK / NAK / STALL / NYET
--------------	---------------------------------

ACK = "Got it, success!"

NAK = "Busy now, try again" (device buffer full)

STALL = "Error! I can't do this" (unsupported request)

NYET = "Not yet" (USB 2.0 High Speed only)

Packet Types in Action

Mouse sends position

1. TOKEN (IN) —————→ Host asks mouse for data
"Endpoint 1, give me your state"
2. DATA (DATA0/DATA1) ← Mouse responds with packet
[0x01, 0x00, 0x05, 0xFF, 0x00]
 - └─ scroll: 0
 - └─ Y: -1 pixel
 - └─ X: +5 pixels
 - └─ X: +5 pixels
 - └─ right button: not pressed
 - └─ left button: pressed
3. HANDSHAKE (ACK) —————→ Host confirms receipt

Transaction: Token + Data + Status

A **transaction** is the fundamental unit of USB communication. It always consists of **3 packets** in sequence:

TRANSACTION (one complete exchange)

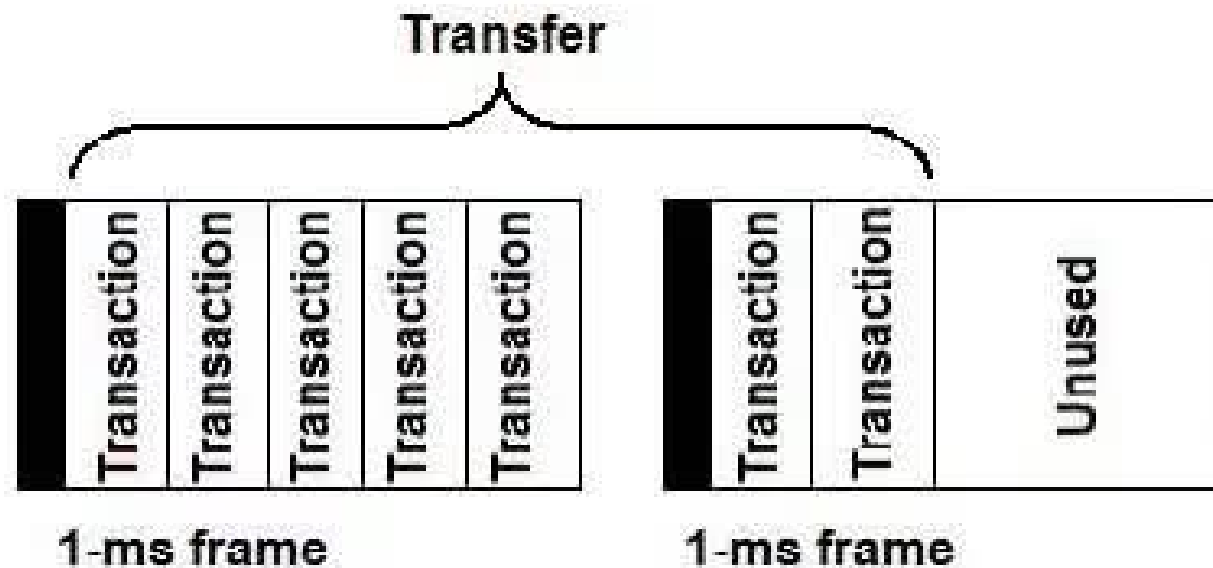
- | | |
|------------------|--------------------------------|
| 1. TOKEN Packet | → Host says who/what/direction |
| 2. DATA Packet | ← Device sends (or host sends) |
| 3. STATUS Packet | → ACK / NAK / STALL |

Think of the transaction like ordering at a counter:

1. **Token** = "Table 3, I'd like a burger" (address + direction)
2. **Data** = Kitchen hands over the burger
3. **Status** = "Thanks, got it!" / "Not ready yet!"

Transfer: One or More Transactions

A **transfer** is a complete logical operation made of one or more transactions.



Example: Copying a 200-byte file from USB flash drive
(max packet size = 64 bytes)

Transfer = 4 transactions:

Transaction 1: Token(IN) → Data(64 bytes) → ACK

Transaction 2: Token(IN) → Data(64 bytes) → ACK

Transaction 3: Token(IN) → Data(64 bytes) → ACK

Transaction 4: Token(IN) → Data(8 bytes) → ACK

↑
short packet signals end

Application sees: 200 bytes received as one read()

Transfer Types vs Transaction

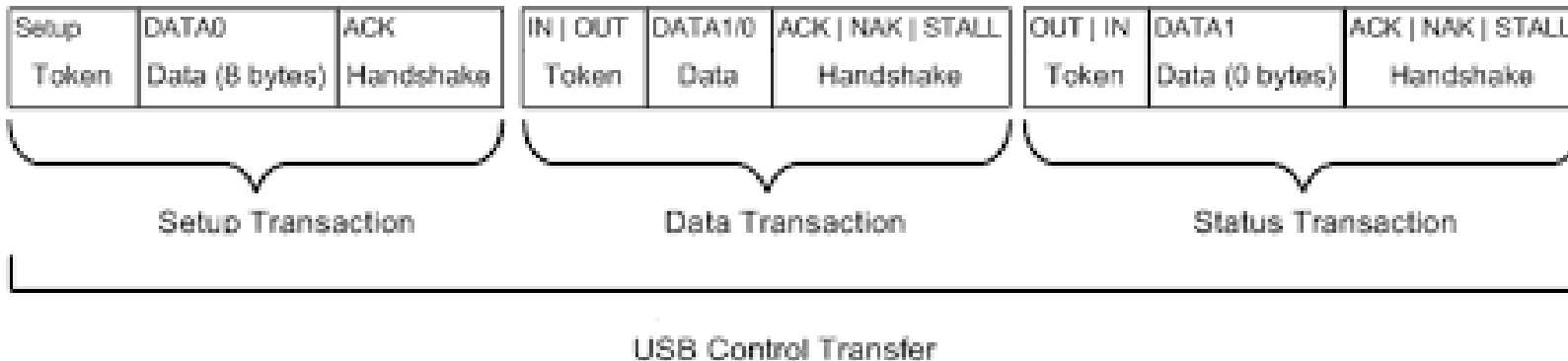
The **transfer type** (Bulk, Interrupt, Isochronous, Control) determines **how transactions are scheduled**:

Control Transfer:

Phase 1: SETUP transaction (always DATA0)

Phase 2: DATA transaction(s) (optional)

Phase 3: STATUS transaction (direction reversed)



Bulk Transfer:

One or more IN/OUT transactions

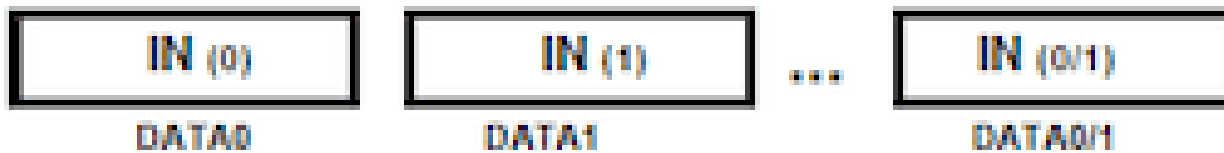
Retried on error (NAK)

No timing guarantee

Bulk
Write



Bulk
Read



Interrupt Transfer:

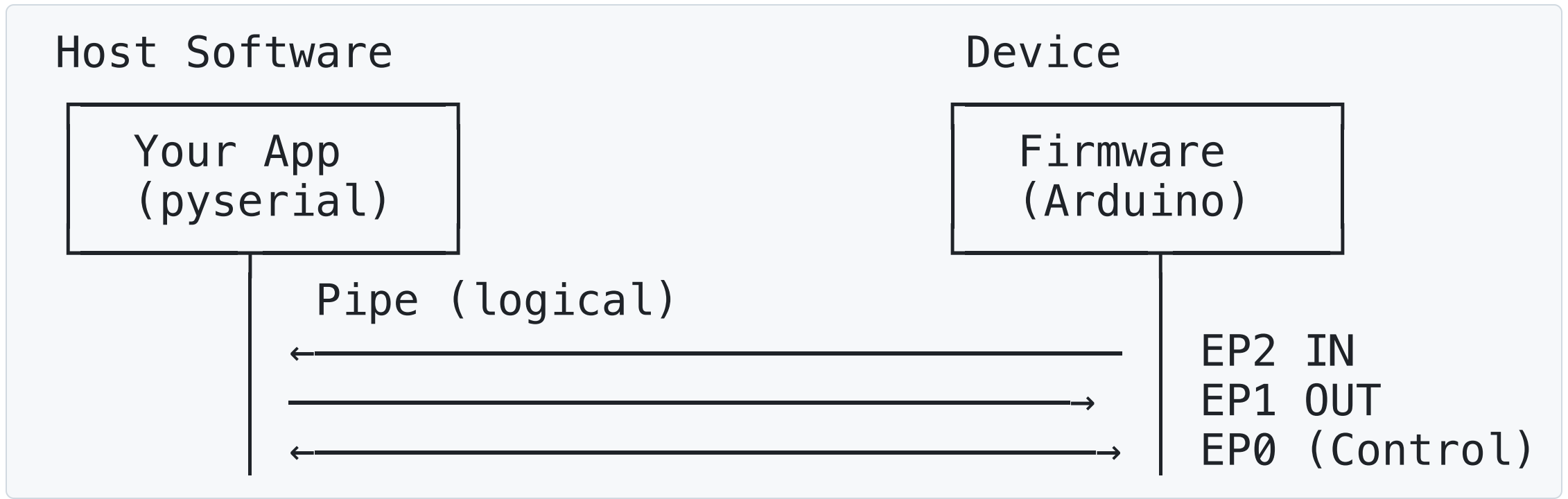
- One transaction per polling interval
- Scheduled by host every N ms

Isochronous Transfer:

- One transaction per frame (1ms)
- No handshake packet – no ACK/NAK!

Pipe: Logical Channel Between Host and Endpoint

A **pipe** is the logical connection between the **host software** and a **device endpoint**.



- A pipe is **not** a wire — it's an abstraction
- Each pipe is bound to exactly **one endpoint**
- The OS and USB driver manage pipes for you

Two Types of Pipes

1. Message Pipe (bidirectional, always EP0)
Used for: Control transfers only
Host can send AND receive on same pipe
Example: GET_DESCRIPTOR, SET_ADDRESS
2. Stream Pipe (unidirectional, EP1–EP15)
Used for: Bulk, Interrupt, Isochronous
Direction fixed: either IN or OUT
Examples:
 EP2 IN → device sends data to host

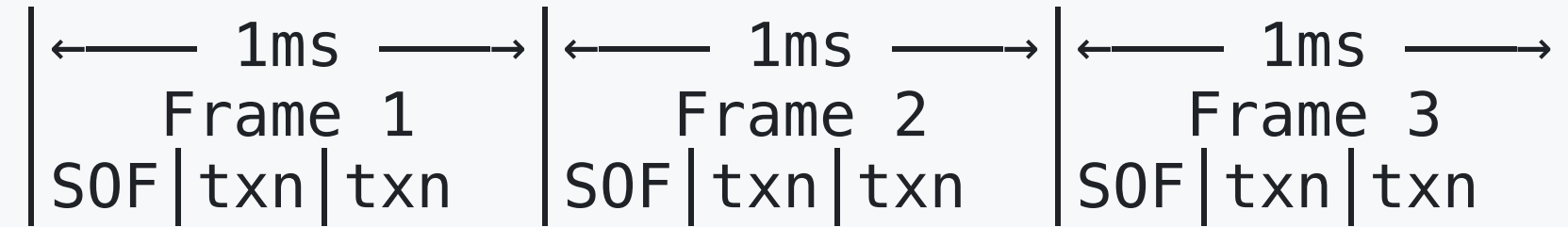
Message Pipe = two-way street. Stream Pipe = one-way street.

Frame: The USB Time Window

USB divides time into **frames** — fixed-length time windows.

USB 2.0 Full Speed: 1 frame = 1ms
USB 2.0 High Speed: 1 microframe = 125μs (8 per frame)

Timeline:



Start-Of-Frame (SOF) packet sent every 1ms by host

SOF (Start-of-Frame): A special token packet the host sends at the beginning of every frame to synchronize all devices.

Frame: What Fits Inside

The host **schedules** transactions into each 1ms frame:

Frame (1ms window)

SOF 8b	Mouse txn 260μs	Keyboard txn 260μs	Bulk txn ... remaining
-----------	--------------------	-----------------------	---------------------------

USB frame scheduling operates on transactions, not transfers, because transactions are the atomic bus-time units that can actually be placed into a frame.

Priority rules:

1. SOF always first
2. Isochronous transactions (reserved slots, no retry)
3. Interrupt transactions (guaranteed within N frames)
4. Control transactions
5. Bulk transactions (fills leftover space)

1 USB Full-Speed frame = 1 ms.

Host divides it like:

```
[ SOF ] [ Iso Tx ] [ Iso Tx ] [ Int Tx ] [ Ctrl Tx ] [ Bulk Tx ] [ Bulk Tx ]
```

Each block = one transaction.

This is why bulk transfers are slowest when the bus is busy — they only get leftover time.

Why “transactions” — not “transfers”?

Why Transfers Can't Be Scheduled?

Transfers may be:

- Large
- Multi-transaction
- Partially complete

Transaction = Smallest schedulable unit
Transfer = Logical data request

Host cannot guarantee: "Finish this whole transfer in this frame"

It can only schedule:

It schedules transactions opportunistically, frame by frame, using whatever time is available.

Example:

Send 512 bytes (Transfer)
→ 8 Transactions
→ 24 Packets

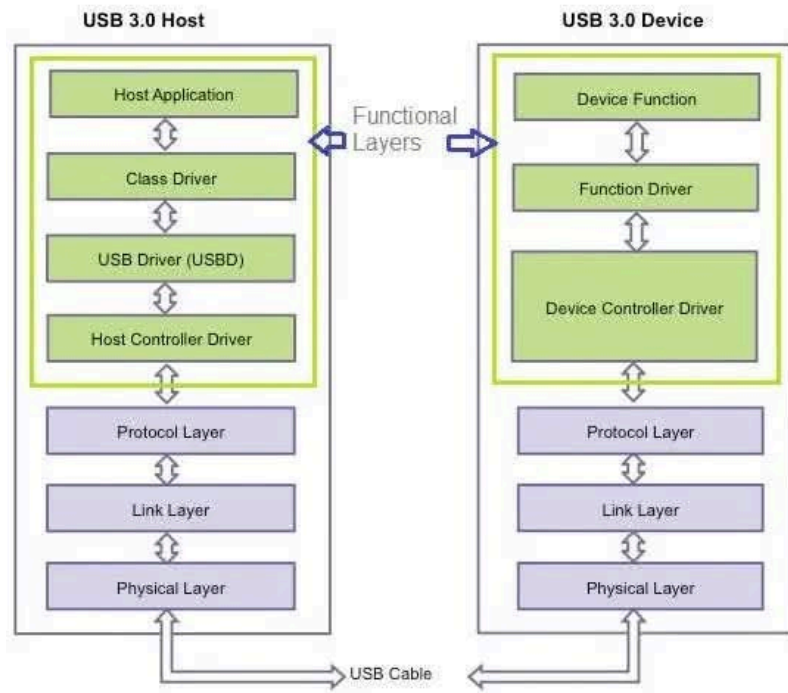
Frame 1 → 2 transactions
Frame 2 → 1 transaction
Frame 3 → 3 transactions
Frame 4 → 0 transactions
Frame 5 → 2 transactions

Only transactions occupy frame slots.

The Full Hierarchy

PIPE	Logical channel: App <=>ndpoint (Message Pipe for EP0, Stream Pipe for rest)
TRANSFER	Complete operation (read file, send command) Made of one or more transactions
TRANSACTION	Token + Data + Status (3 packets) One exchange with one endpoint
PACKET	Smallest unit: SYNC + PID + payload + CRC
FRAME	1ms time window; host schedules all txns

USB Protocol Stack



A **protocol stack** is a set of layers, each with a specific job.

Think of sending a **package overseas**:

Layer	Analogy
Application	You write a letter
Protocol	Post office wraps and labels it
Data Link	Truck driver follows delivery rules
Physical	The actual road and truck

Each layer **only talks to the layer above/below it**.

Layer 1: Physical Layer

Job: Move raw bits (0s and 1s) over a physical medium.

What it includes:

- Connectors: Type-A, Type-B, Type-C, Micro, Mini
- Wires: VBUS (5V), GND, D+, D- (and extra pairs for USB 3.x)
- Voltage levels: 0V = bit 0, 3.3V = bit 1
- Speed: USB 2.0 = 480 Mbps, USB 3.0 = 5 Gbps

Analogy: The physical road and the truck.

It doesn't know what's inside the package — just moves bits.

Host (Computer)	Device (Arduino)
[D+]	[D+]
[D-]	[D-]
[VBUS 5V]	[VBUS]
[GND]	[GND]

USB 2.0: uses D+ / D- (differential signaling)

USB 3.0: adds TX+/TX- and RX+/RX- pairs

Key point: Differential signaling (D+ and D-) reduces noise.
If both wires are interfered equally, the difference stays the same.

Layer 2: Data Link Layer

Job: Package raw bits into **frames/packets**, detect and handle errors.

What it includes:

- Breaks data into packets with headers and checksums (CRC)
- Detects transmission errors (but USB retries, unlike Ethernet)
- Manages **start-of-frame (SOF)** signals every 1ms
- Controls when a device is allowed to talk (token-based)

Analogy: The delivery driver who follows traffic rules, checks the package isn't damaged, and signs for delivery.

Token-based communication:

USB uses a **token-based** system. Only one device talks at a time.

Host sends TOKEN packet → "Device #5, endpoint 1, READ"

↓

Device #5 responds with DATA packet

↓

Host sends HANDSHAKE → ACK (success) or NAK (busy/error)

No device can talk without being asked.

This is why USB is a **polled** bus, not event-driven.

Layer 3: Protocol Layer (USB Class Layer)

Job: Define **what the data means** and how devices communicate at a high level.

This is where **USB Device Classes** live:

Class	Protocol	Device Examples
CDC	Communications Device Class	Arduino serial, Modem
HID	Human Interface Device	Keyboard, Mouse, Gamepad
MSC	Mass Storage Class	USB Flash Drive, SD reader
UAC	USB Audio Class	Headset, Microphone
UVC	USB Video Class	Webcam

Arduino Example (CDC Protocol):

Arduino firmware declares itself as CDC device

↓

Host OS sees: "This is a virtual serial port"

↓

OS creates /dev/ttyUSB0 (Linux) or COM3 (Windows)

↓

Your program can read/write as if it's a serial port

The CDC protocol defines the exact format of **control requests, data format, and line coding** (baud rate, stop bits).

Keyboard Example (HID Protocol)

Every key press is reported as a structured **HID Report**:

HID Report (8 bytes for keyboard):

Modifier (Ctrl, Shift...)	Reserved (0x00)	Key1 – Key6 (Key codes being pressed)
---------------------------------	--------------------	--

Example: pressing Shift + A

→ [0x02, 0x00, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00]
 Shift 'a'

Notice that the application layer (your Python code) doesn't care about packets, CRC, or voltage levels. It just calls an API — the stack handles the rest.

```
import hid # pip install hidapi

# Find a HID device (e.g., USB keyboard/gamepad)
device = hid.device()
device.open(0x046D, 0xC52B) # Logitech VID/PID

device.set_nonblocking(1)

print("Reading HID reports...")
while True:
    report = device.read(64) # Read up to 64 bytes
    if report:
        print(f"HID Report: {report}")
        # report[0] = modifier keys (Shift, Ctrl, Alt)
        # report[2] = key code being pressed

device.close()
```

Layer 4: Application Layer

Job: The actual program the **user or developer** writes to use the device.

This is where **you** work most of the time.

Examples:

- A Python script that reads temperature from an Arduino
- A game that reads joystick input via HID
- A file manager that copies files to a USB drive (MSC)
- A video conferencing app that captures webcam frames (UVC)

Key point: Application layer doesn't care about packets, CRC, or voltage.

It just calls an API — the stack handles the rest.

pyusb Example

```
import usb.core
import usb.util

# Find device by Vendor ID and Product ID
dev = usb.core.find(idVendor=0x2341, idProduct=0x0043) # Arduino Uno

if dev is None:
    raise ValueError("Arduino not found!")

# Set active configuration (Layer 3 setup)
dev.set_configuration()

# Get the first endpoint (Layer 3/2 details hidden from us)
cfg = dev.get_active_configuration()
intf = cfg[(0, 0)]
ep = usb.util.find_descriptor(
    intf,
    custom_match=lambda e:
        usb.util.endpoint_direction(e.bEndpointAddress)
        == usb.util.ENDPOINT_IN
)

# Read data – Application layer just calls read()
data = dev.read(ep.bEndpointAddress, ep.wMaxPacketSize)
print(f"Received: {bytes(data)}")
```

High Level API

Most of the time, the application layer is **even simpler**:

```
# Using pyserial – Application layer at its simplest
import serial

ser = serial.Serial('/dev/ttyUSB0', 9600)

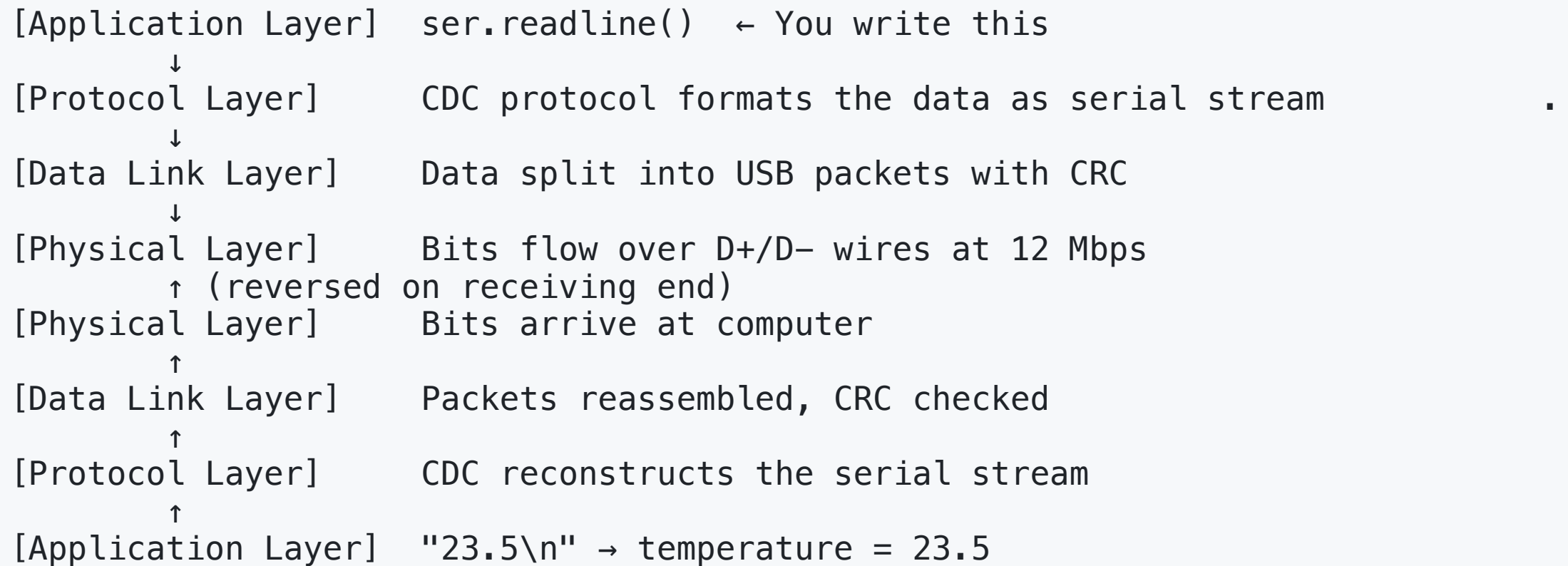
# You never think about:
# – Physical Layer: D+/D– voltage, connectors
# – Data Link Layer: Packets, CRC, token bus
# – Protocol Layer: CDC line coding, control requests

# You just do this:
temperature = float(ser.readline().decode().strip())
print(f"Temperature: {temperature}°C")
```

All 3 lower layers are invisible to the application developer.

All 4 Layers Working Together

Scenario: You read a temperature value from an Arduino sensor.



Why Layers Matter

Benefit	Explanation
Abstraction	You use <code>serial.read()</code> , not raw USB packets
Replaceability	Swap USB 2.0 for USB 3.0 — application code unchanged
Reusability	CDC protocol works for Arduino, modems, GPS modules
Debugging	Problems can be isolated to one layer

Example: If your Arduino data is corrupted → suspect Data Link (CRC error) or Physical (bad cable), not your Python code.

Summary: USB Protocol Stack

Layer 4: Application	<code>ser.readline()</code> , <code>hid.read()</code>
Layer 3: Protocol	CDC, HID, MSC, UAC, UVC
Layer 2: Data Link	Packets, Token bus, CRC, ACK
Layer 1: Physical	D+/D-, connectors, 5V, speed

Rule: Each layer only knows about its neighbors.

Lower layers serve upper layers.

Upper layers are simpler because lower layers hide complexity.